

NM 补充资料 V3

导入模块 Import

每道题先把下面这段复制到第一个 cell:

代码块

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy.interpolate import lagrange
4  from scipy.linalg import lu, solve
5  import scipy.optimize as sop
6
7  plt.rcParams['font.size'] = 12
```

Chapter 0: 收敛分析 Convergence Analysis

几乎每道数值方法题最后都会让你做收敛分析

Order / 阶数	Error ratio when step halved / 步长减半时误差比值	Meaning / 含义
1st order	≈ 2.0 / 误差减半	Forward/Backward Euler, Forward Difference
2nd order	≈ 4.0 / 误差变 1/4	Central Difference, Heun, Midpoint, Trapezoid
4th order	≈ 16.0 / 误差变 1/16	Simpson's, 4th-order Central Difference
6th order	≈ 64.0 / 误差变 1/64	Weddle's rule

收敛分析步骤

代码块

```
1  # [REPLACE: exact_value - copy exact value from the exam question / 替换: 从考题复制精确值]
2  exact_value = 1.0
```

```

3
4 # Step sizes: halve each time / 步长每次减半
5 h_values = np.array([0.1, 0.05, 0.025, 0.0125, 0.00625])
6
7 errors = []
8 for h in h_values:
9     approx = my_method(h) # [REPLACE: call your implemented function / 替
    换: 调用你实现的函数]
10     err = np.abs(approx - exact_value)
11     errors.append(err)
12
13 # Plot log-log / 画 log-log 图
14 plt.loglog(h_values, errors, 'o-', label='Method')
15 plt.xlabel('Step size h'); plt.ylabel('Absolute error')
16 plt.legend(); plt.grid(True, which='both', ls='--')
17 plt.show()
18
19 # Compute ratios / 计算比值
20 for i in range(len(errors)-1):
21     ratio = errors[i] / errors[i+1]
22     order = np.log2(ratio)
23     print(f"h={h_values[i]:.5f} -> {h_values[i+1]:.5f}: ratio={ratio:.2f},
    order={order:.2f}")

```

收敛分析结果

Ratio ≈	Sentence to copy into answer / 抄写至答题区	Order
2.0	"Halving the step size reduces the error by a factor of ~2, indicating first-order accuracy $O(h)$."	1st
4.0	"Halving the step size reduces the error by a factor of ~4, indicating second-order accuracy $O(h^2)$."	2nd
16.0	"Halving the step size reduces the error by a factor of ~16, indicating fourth-order accuracy $O(h^4)$."	4th
64.0	"Halving the step size reduces the error by a factor of ~64, indicating sixth-order accuracy $O(h^6)$."	6th

文字题模板

Why can error stop decreasing when Δx is very small? / 为什么 Δx 很小时误差不再下降?

"When Δx becomes very small, for example below 10^{-8} , round-off errors from finite-precision arithmetic can dominate over truncation errors. Therefore, the total error may stop decreasing, flatten out, or even increase. This should be mentioned when discussing the log-log error plot."

(当 Δx 极小, 例如小于 10^{-8} 时, 有限精度计算中的 round-off errors 可能超过 truncation errors。因此, 总误差可能不再下降, 而是 flatten out, 甚至 increase。在讨论 log-log error plot 时需要提到这一点。)

Chapter 1: Interpolation & Curve Fitting 插值与曲线拟合

识别标志

The exam gives data points and asks you to interpolate (exact fit) or curve-fit (approximate), compare methods, or explain overfitting.

(题目给数据点, 让你插值 (精确过点) 或拟合 (近似), 比较方法, 或解释过拟合。)

真题原文

2020 Q1: "Calculate the Lagrange polynomial... Fit a 3rd degree polynomial... Explain how you know in advance what degree of polynomial you need..."

2022 Q1: "Explain what the three techniques of (i) interpolation, (ii) extrapolation and (iii) curve-fitting are..."

2023 Q1: "Explain how you know mathematically... what degree polynomial can be used to exactly interpolate this data..."

2024 Q1: "Demonstrate how Lagrange polynomial interpolation... can exactly fit... Demonstrate how if you choose too few sample points..."

2025 Q1: "Generate a sample data set of $N=10$ data point pairs... add some random noise... construct the Lagrange polynomial interpolant... least squares polynomial curve-fitting..."

题目会给什么

- Data points (x, y) , e.g. $(0.0, 0.5), (0.2, 0.4), \dots$
- Sometimes a specific function to sample from (e.g. $g(x) = \cos(2\pi x)$ in 2025)
- Sometimes noise/randomness to add
- Requested polynomial degree for fitting

Worked Example (from 2020 Q1) / 例题

Given: $(0.0, 0.5), (0.2, 0.4), (0.5, 0.3), (0.65, 0.5), (0.8, 0.9), (1.0, 0.8)$

Part A: Lagrange Interpolation (exact fit) / Lagrange 插值

代码块

```
1  xi = np.array([0.0, 0.2, 0.5, 0.65, 0.8, 1.0]) # [REPLACE: copy x from exam / 替换: 从考题复制 x]
2  yi = np.array([0.5, 0.4, 0.3, 0.5, 0.9, 0.8]) # [REPLACE: copy y from exam / 替换: 从考题复制 y]
3
4  poly = lagrange(xi, yi)
5  degree = len(poly.c) - 1
6  print(f"Degree: {degree}") # n points -> degree n-1 / n 个点对应 n-1 次多项式
7
8  x_new = np.linspace(xi.min(), xi.max(), 200)
9  plt.plot(xi, yi, 'ko', label='Data')
10 plt.plot(x_new, poly(x_new), 'r-', label='Lagrange')
11 plt.xlabel('x'); plt.ylabel('y'); plt.legend(); plt.show()
```

Part B: Curve Fitting with Different Degrees / 不同次数拟合

代码块

```
1  for deg in range(0, 11):
2      coeffs = np.polyfit(xi, yi, deg)
3      p = np.poly1d(coeffs)
4      y_fit = p(xi)
5      sse = np.sum((yi - y_fit)**2)
6      print(f"Degree {deg:2d}: SSE = {sse:.6e}")
```

本题文字部分注意:

"The error drops significantly from degree 0 to degree 1, but shows only minimal improvement beyond degree 1. Therefore, degree 1 is sufficient, while higher degrees may lead to overfitting."

(误差从 degree 0 到 degree 1 明显下降, 但超过 degree 1 后提升很小。因此 degree 1 已经足够, 而更高次数可能导致 overfitting。)

Worked Example (from 2025 Q1 — Noisy Data) / 带噪声数据例题

Given: $g(x) = \cos(2\pi x)$, sample $N=10$ points in $[0,1]$, add noise with $\text{std}=0.1$.

代码块

```
1  N = 10 # [REPLACE: N from exam / 替换: 从考题复制 N]
2  x = np.linspace(0, 1, N) # [REPLACE: interval from exam / 替换: 从考题复制区间]
```

```

3  g = lambda x: np.cos(2 * np.pi * x)          # [REPLACE: function from exam / 替
      换: 从考题复制函数]
4  np.random.seed(42)
5  noise = 0.1 * np.random.randn(N)             # [REPLACE: noise std from exam /
      替换: 从考题复制噪声标准差]
6  y_noisy = g(x) + noise
7
8  # Lagrange interpolation on noisy data / 对带噪声数据做 Lagrange 插值
9  poly_lag = lagrange(x, y_noisy)
10 x_fine = np.linspace(0, 1, 200)
11 plt.plot(x, y_noisy, 'ko', label='Noisy data')
12 plt.plot(x_fine, g(x_fine), 'g--', label='True g(x)')
13 plt.plot(x_fine, poly_lag(x_fine), 'r-', label='Lagrange')
14 plt.legend(); plt.show()
15
16 # Least-squares fits of degree 1, 3, 5 / 最小二乘拟合 1/3/5 次
17 for deg in [1, 3, 5]:
18     coeffs = np.polyfit(x, y_noisy, deg)
19     p = np.poly1d(coeffs)
20     # MSE against noisy data points / 对噪声数据点的 MSE
21     mse_noisy = np.mean((y_noisy - p(x))**2)
22     # MSE against true function (fine grid) / 对真实函数的 MSE
23     mse_true = np.mean((g(x_fine) - p(x_fine))**2)
24     print(f"Deg {deg}: MSE noisy={mse_noisy:.4f}, MSE true={mse_true:.4f}")

```

更优算法: Newton Interpolation (Divided Differences) / Newton 差商插值

Newton form builds the polynomial recursively using divided differences. It is equivalent to Lagrange but numerically more efficient for adding new points.

(Newton 形式用差商递归构造多项式, 与 Lagrange 等价但增加新点时数值效率更高。)

代码块

```

1  def newton_poly_coeffs(xi, yi):
2      """Compute divided-difference coefficients / 计算差商系数"""
3      a = yi.copy()
4      N = len(a) - 1
5      for k in range(1, N+1):
6          for i in range(k, N+1):
7              a[i] = (a[i] - a[k-1]) / (xi[i] - xi[k-1])
8      return a
9
10 def eval_newton_poly(a, xi, x):
11     """Evaluate Newton polynomial at x / 在 x 处求值"""
12     N = len(a) - 1
13     p = a[N]
14     for k in range(1, N+1):
15         p = a[N-k] + (x - xi[N-k]) * p

```

```
16         return p
17
18     # [REPLACE: copy xi, yi from exam / 替换: 从考题复制 xi, yi]
19     a = newton_poly_coeffs(xi, yi)
20     y_new = eval_newton_poly(a, xi, x_new)
```

文字题模板

Interpolation vs Extrapolation vs Curve Fitting / 插值、外推与曲线拟合的区别

"Interpolation constructs a function that passes exactly through all known data points and estimates values within the data range. Extrapolation extends that function beyond the data bounds, but accuracy usually drops quickly with distance. Curve fitting finds an approximating function that captures the overall trend without necessarily passing through every point; it minimises SSE and is preferred for noisy data."

(Interpolation 构造一个精确通过所有已知数据点的函数，用来估计数据范围内的值。Extrapolation 把这个函数延伸到数据范围之外，但距离越远准确性通常下降越快。Curve fitting 寻找一个近似函数来捕捉整体趋势，不一定通过每个点；它最小化 SSE，更适合 noisy data。)

How many degrees are needed to fit n points exactly? / 精确拟合 n 个点需要几次多项式?

"For n distinct data points, there exists a unique polynomial of degree at most n-1 that interpolates them exactly. A degree-(n-1) polynomial has n unknown coefficients, and n data points provide n constraint equations."

(对于 n 个不同的数据点，存在唯一一个次数最多为 n-1 的多项式可以精确插值这些点。degree-(n-1) polynomial 有 n 个未知系数，而 n 个数据点提供 n 个约束方程。)

What does polyfit minimise? / polyfit 会将什么值最小化?

"It minimises the sum of squared errors (SSE) between the fitted polynomial and the data points, equivalently related to the root-mean-square (RMS) error."

(它最小化 fitted polynomial 和 data points 之间的 sum of squared errors (SSE)，也就是与 root-mean-square (RMS) error 直接相关的误差。)

Why might curve-fitting be preferred over interpolation for noisy data? / 噪声数据为什么优先选拟合?

"Interpolation forces the polynomial to pass through every noisy data point, thereby fitting the noise rather than the underlying signal. This produces large oscillations between points and poor generalisation. Curve-fitting with a lower-degree polynomial captures the true trend while leaving the random noise unexplained, yielding a model that generalises better to the true function."

(插值强制多项式过每个噪声点，从而拟合了噪声而非信号，产生振荡且泛化差。低次曲线拟合捕捉真实趋势而忽略随机噪声，泛化更好。)

Why are composite rules better? / 为什么 composite rules 更好?

"Composite rules avoid the oscillations caused by fitting one single high-degree polynomial over a large interval. When refining the grid, previously computed function values at shared nodes can also be reused, reducing redundant function evaluations."

(Composite rules 可以避免在大区间上拟合单个高次多项式导致的 oscillations。细化 grid 时, shared nodes 上之前已经计算过的函数值也可以重复使用, 从而减少 redundant function evaluations。)

Is a 10th-order polynomial always better? / 10次多项式一定更好吗?

"No. Although the 10th-order polynomial may give a lower SSE on the given data points, it is likely overfitting. The high-degree polynomial can oscillate strongly between data points and give unreliable extrapolation outside the data range. A lower-degree model is often better because it captures the true trend without fitting the noise."

(不一定。虽然 10th-order polynomial 可能在给定数据点上有更低的 SSE, 但它很可能是 overfitting。高次多项式可能在数据点之间剧烈振荡, 并且在数据范围外的 extrapolation 不可靠。较低次数的模型通常更好, 因为它能捕捉真实趋势, 而不是拟合噪声。)

What is Runge's Phenomenon? / 什么是 Runge 振荡现象?

"Runge's Phenomenon is the oscillation that can appear near the edges of an interval when using high-degree polynomial interpolation, even for a smooth function. A classic example is $f(x) = 1/(1 + 25x^2)$ on $[-1, 1]$. With a high-degree polynomial, the fit often becomes poor near the boundaries. Therefore, piecewise low-degree interpolation or composite methods are usually preferred."

(Runge 振荡现象是指: 即使函数本身光滑, 使用高次多项式插值时, 区间边缘附近仍可能出现明显振荡。经典例子是区间 $[-1, 1]$ 上的 $f(x) = 1/(1 + 25x^2)$ 。当使用高次多项式时, 边界附近的拟合常常会变差。因此, 通常更倾向于使用分段低次插值或 composite methods。)

Chapter 2: Numerical Differentiation 数值微分

识别标志

The exam gives $f(x)$ and x_0 , asks for derivative via finite differences and convergence analysis.
(题目给 $f(x)$ 和 x_0 , 让你用有限差分计算导数并做收敛分析。)

真题原文

2020 Q2.1: "Use forward and central finite difference methods to compute the derivative of $f(x) = \cos(x) + x^2$ at $x=0.75$..."

2022 Q2: "Three possible methods: first-order forward, second-order central, fourth-order central..."

2024 Q2: "Explain why we need to discretise... Explain the difference between forward, backward and central... Perform Taylor series analysis..."

2025 Q2: "Explain why estimating derivatives numerically is often necessary... Explain the difference between forward and central... Use Taylor series expansions to derive formulae... $f(x) = x^3 - 2x^2 + 5$ at $x=1$..."

题目会给什么

- $f(x)$ (e.g. $\cos(x) + x^2$, $x^3 - 2x^2 + 5$, $\sin(x)$)
- x_0 (e.g. 0.75 , 1 , $\pi/2$)
- Exact derivative (given or computed by hand)

All Three Difference Methods / 三种差分方法

代码块

```
1  # [REPLACE: f - copy from exam, e.g. lambda x: np.cos(x) + x**2 / 替换: 从考题
   复制 f, 例如 lambda x: np.cos(x) + x**2]
2  # [REPLACE: df_exact - analytical derivative, e.g. lambda x: -np.sin(x) +
   2*x / 替换: 解析导数, 例如 lambda x: -np.sin(x) + 2*x]
3  f = lambda x: np.cos(x) + x**2
4  df_exact = lambda x: -np.sin(x) + 2*x
5
6  def forward_diff(f, x, h):
7      """First-order O(h) / 一阶精度"""
8      return (f(x + h) - f(x)) / h
9
10 def backward_diff(f, x, h):
11     """First-order O(h) / 一阶精度"""
12     return (f(x) - f(x - h)) / h
13
14 def central_diff(f, x, h):
15     """Second-order O(h^2) / 二阶精度"""
16     return (f(x + h) - f(x - h)) / (2 * h)
17
18 def central_diff_4th(f, x, h):
19     """Fourth-order O(h^4) / 四阶精度 (2022 Q2)"""
20     return (-f(x + 2*h) + 8*f(x + h) - 8*f(x - h) + f(x - 2*h)) / (12 * h)
21
22 x0 = 0.75 # [REPLACE: x0 from exam / 替换: 从考题复制 x0]
23 exact_val = df_exact(x0)
```

Taylor Series Derivations (for written questions) / Taylor 展开推导

Forward Difference derivation / 前向差分推导:

"Taylor expansion: $f(x+h) = f(x) + hf'(x) + (h^2/2)f''(x) + O(h^3)$. Rearranging: $[f(x+h) - f(x)]/h = f'(x) + O(h)$. Therefore the forward difference is first-order accurate."

Central Difference derivation / 中心差分推导：

" $f(x+h) = f(x) + hf'(x) + (h^2/2)f''(x) + (h^3/6)f'''(x) + O(h^4)$
 $f(x-h) = f(x) - hf'(x) + (h^2/2)f''(x) - (h^3/6)f'''(x) + O(h^4)$
Subtracting: $f(x+h) - f(x-h) = 2hf'(x) + O(h^3)$
Therefore $[f(x+h) - f(x-h)]/(2h) = f'(x) + O(h^2)$."

Backward Difference derivation / 后向差分推导：

" $f(x-h) = f(x) - hf'(x) + (h^2/2)f''(x) + O(h^3)$. Rearranging: $[f(x) - f(x-h)]/h = f'(x) + O(h)$."

结果解读

Method / 方法	Ratio / 比值	Order / 阶数	Sentence to copy / 抄写
Forward	≈ 2.0	1st $O(h)$	"Forward difference is first-order $O(h)$."
Backward	≈ 2.0	1st $O(h)$	"Backward difference is first-order $O(h)$."
Central (2nd)	≈ 4.0	2nd $O(h^2)$	"Central difference is second-order $O(h^2)$."
Central (4th)	≈ 16.0	4th $O(h^4)$	"Fourth-order central difference is $O(h^4)$."

文字题模板

Why do we need to discretise? / 为什么需要离散化？

"Computers can only store and operate on finite amounts of data. A continuous function $f(x)$ is defined at infinitely many points, which cannot be represented exactly in digital memory. Discretisation replaces the continuous domain with a finite mesh or grid of points, allowing us to compute approximations using a finite number of arithmetic operations."
(计算机只能存储有限数据。连续函数 $f(x)$ 在无穷多点有定义，无法被数字内存精确表示。离散化用有限网格点代替连续域，使我们能用有限次运算计算近似值。)

Why is central difference more accurate than forward difference? / 为什么中心差分比前向差分更准确？

Forward difference has leading error term $h f''(x)/2$, so its error is $O(h)$ and it is first-order accurate. Central difference uses values on both sides of x , so the first-order error terms cancel by symmetry. After Taylor expansion, its leading error term is $h^2 f'''(x)/6$, so its error is $O(h^2)$ and it is second-order accurate.
Forward difference 的 leading error term 是 $h f''(x)/2$ ，所以 error 是 $O(h)$ ，属于 first-order accurate。Central difference 使用 x 两侧的 values，因此 first-order error terms 会因为 symmetry

抵消。通过 Taylor expansion 后，它的 leading error term 是 $h^2 f''(x)/6$ ，所以 error 是 $O(h^2)$ ，属于 second-order accurate。

How do Taylor expansions derive forward and central difference errors? / 如何用 Taylor expansion 推导 forward 和 central difference 的误差?

Taylor expansion explains the accuracy of finite difference formulae. For forward difference, expanding $f(x+h)$ around x gives $f(x+h) = f(x) + h f'(x) + h^2 f''(x)/2 + \dots$. After rearranging, $[f(x+h)-f(x)]/h = f'(x) + h f''(x)/2 + O(h^2)$, so the leading error term is $h f''(x)/2$ and the method is first-order accurate, with error $O(h)$.

Taylor expansion 可以解释 finite difference formulae 的精度。对于 forward difference，把 $f(x+h)$ 在 x 附近展开，得到 $f(x+h) = f(x) + h f'(x) + h^2 f''(x)/2 + \dots$ 。整理后， $[f(x+h)-f(x)]/h = f'(x) + h f''(x)/2 + O(h^2)$ ，所以 leading error term 是 $h f''(x)/2$ ，该方法是 first-order accurate，误差为 $O(h)$ 。

Why can the error increase when Δx is very small? / 为什么 Δx 很小时误差反而会增加?

"For moderate Δx , decreasing Δx reduces the truncation error, so the numerical derivative becomes more accurate. However, when Δx is extremely small, subtracting nearly equal function values causes loss of significant digits. At this stage, round-off error dominates, so the total error may level off or increase instead of following the theoretical order of convergence."

(在中等大小的 Δx 下，减小 Δx 会降低 truncation error，所以数值导数更准确。但当 Δx 极小时，两个非常接近的函数值相减会造成 significant digits loss。此时 round-off error 占主导，所以总误差可能 flatten out 或 increase，而不是继续按理论收敛阶下降。)

Chapter 3: Numerical Integration 数值积分

识别标志

The exam gives $f(x)$ and limits $[a, b]$, asks for Midpoint / Trapezoidal / Simpson's / Weddle's rule and convergence analysis.

(题目给 $f(x)$ 和 $[a,b]$ ，让你实现积分方法并做收敛分析。)

真题原文

2020 Q3: "Composite integration rules... approximate using composite midpoint, trapezoid and Simpson rules..."

2023 Q3: "Write a single function... Midpoint, Trapezoid, Simpson's, Weddle's rule... convergence analysis..."

2024 Q3: "Consider the function $f(x)$ given by code... interval $[-1,1]$... exact integral $I = 2.05884510697215$... non-uniform discretisation..."

2025 Q3: " $f(x) = x^2 + 1$ over $[1,3]$... determine exact analytical value... Midpoint, Trapezoid, Simpson's..."

题目会给什么

- $f(x)$ (e.g. $\cos(x)$, $x^2 + 1$, or code-defined function)
- a and b (e.g. 0 to $\pi/2$, 1 to 3, -1 to 1)
- Exact integral value (sometimes given, sometimes compute analytically)

Universal Integration Function / 通用积分函数

代码块

```
1  def integrate(f, a, b, n, rule):
2      """
3      Numerical integration with selectable rule.
4      数值积分通用函数。
5
6      f: integrand [REPLACE: copy from exam]
7      a, b: limits [REPLACE: copy from exam]
8      n: number of sub-intervals (must be EVEN for Simpson & Weddle)
9      rule: 1=Midpoint, 2=Trapezoidal, 3=Simpson's, 4=Weddle's
10     """
11     h = (b - a) / n
12     x = np.linspace(a, b, n + 1)
13
14     if rule == 1: # Midpoint / 中点法
15         x_mid = np.linspace(a + h/2, b - h/2, n)
16         return h * np.sum(f(x_mid))
17
18     elif rule == 2: # Trapezoidal / 梯形法
19         return h * (0.5*f(x[0]) + np.sum(f(x[1:-1])) + 0.5*f(x[-1]))
20
21     elif rule == 3: # Simpson's / Simpson 法
22         if n % 2 != 0:
23             raise ValueError("n must be even for Simpson's rule")
24         return (h/3) * (f(x[0]) + f(x[-1])
25                        + 4*np.sum(f(x[1:-1:2]))
26                        + 2*np.sum(f(x[2:-1:2])))
27
28     elif rule == 4: # Weddle's (Richardson on Simpson) / Weddle 法
29         if n % 2 != 0:
30             raise ValueError("n must be even for Weddle's rule")
31         S_n = integrate(f, a, b, n, 3)
32         S_2n = integrate(f, a, b, 2*n, 3)
33         return S_2n + (S_2n - S_n) / 15
34
35     else:
36         raise ValueError("rule must be 1, 2, 3, or 4")
```

Non-Uniform Discretisation / 非均匀离散化

If the function varies rapidly near $x=0$ but slowly near ± 1 (as in 2024 Q3), use smaller sub-intervals where the function changes quickly.

(如果函数在 $x=0$ 附近变化快而在 ± 1 附近变化慢，在变化快的地方用更小子区间。)

代码块

```
1 def integrate_nonuniform(f, x_nodes):
2     """
3     Composite trapezoidal rule with non-uniform spacing.
4     非均匀间距的复合梯形法。
5     x_nodes: array of node positions (not necessarily equally spaced)
6             [REPLACE: copy your non-uniform node positions]
7     """
8     total = 0.0
9     for i in range(len(x_nodes) - 1):
10         h_i = x_nodes[i+1] - x_nodes[i]
11         total += 0.5 * h_i * (f(x_nodes[i]) + f(x_nodes[i+1]))
12     return total
13
14 # Example: finer grid near x=0 / 示例: x=0 附近加密
15 x_fine = np.concatenate([
16     np.linspace(-1, -0.2, 5),
17     np.linspace(-0.2, 0.2, 10), # fine near 0 / 0 附近加密
18     np.linspace(0.2, 1, 5)
19 ])
20 I_nonuniform = integrate_nonuniform(f, x_fine)
```

Worked Example (2025 Q3) / 例题

Given: $f(x) = x^2 + 1$, integrate from 1 to 3. Exact: $I = [x^3/3 + x]_1^3 = (9+3) - (1/3+1) = 32/3 \approx 10.6667$.

代码块

```
1 f = lambda x: x**2 + 1 # [REPLACE: copy from exam / 替换: 从考题复制]
2 a, b = 1.0, 3.0 # [REPLACE: copy limits from exam / 替换: 从考题复制积分限]
3 exact = 32.0 / 3.0 # [REPLACE: exact value or compute analytically / 替换: 精确值或解析计算]
4
5 n_values = np.array([2, 4, 8, 16, 32, 64])
6 for n in n_values:
7     I_mid = integrate(f, a, b, n, 1)
8     I_trap = integrate(f, a, b, n, 2)
9     I_simp = integrate(f, a, b, n, 3)
```

```
10 print(f'n={n:3d}: Mid={abs(I_mid-exact):.10f}, Trap={abs(I_trap-  
exact):.10f}, Simp={abs(I_simp-exact):.10f}')
```

本题文字部分注意：

"For this specific function $f(x) = x^2 + 1$, Simpson's rule is actually exact (error $\sim$ machine precision) because Simpson's rule is exact for polynomials up to degree 3. The Midpoint and Trapezoidal rules still show their expected $O(h^2)$ convergence because the function is not a constant or linear."

(对于 $f(x)=x^2+1$ ，Simpson 法则实际上是精确的（误差约机器精度），因为 Simpson 法则对次数 ≤ 3 的多项式精确。中点法和梯形法仍显示预期的 $O(h^2)$ 收敛，因为函数不是常数或线性的。)

结果解读

Method / 方法	Ratio / 比值	Order / 阶数
Midpoint	≈ 4.0	2nd $O(h^2)$
Trapezoidal	≈ 4.0	2nd $O(h^2)$
Simpson's	≈ 16.0	4th $O(h^4)$
Weddle's	≈ 64.0	6th $O(h^6)$

文字题模板

Why use composite rules? / 为什么使用 Composite rules?

"Composite rules reduce redundant function evaluations by reusing shared interior points between sub-intervals. They also avoid the poor behaviour of using one single high-degree polynomial over a large interval, such as wild oscillations near the boundaries."

(Composite rules 通过重复利用子区间之间共享的内部点，减少不必要的函数计算。它们也避免了在大区间上使用单个高次多项式带来的差表现，例如边界附近的剧烈振荡。)

Why is $(4I_{h/2} - I_h)/3$ more accurate? / 为什么 Richardson 外推更准确?

"If the leading error in the trapezoidal rule is $O(h^2)$, then $I_h \approx I + ah^2$ and $I_{h/2} \approx I + ah^2/4$. Multiplying $I_{h/2}$ by 4 and subtracting I_h cancels the ah^2 term, leaving an $O(h^4)$ estimate."

(如果 trapezoidal rule 的主导误差是 $O(h^2)$ ，那么 $I_h \approx I + ah^2$ ， $I_{h/2} \approx I + ah^2/4$ 。将 $I_{h/2}$ 乘以 4 再减去 I_h ，可以抵消 ah^2 项，得到 $O(h^4)$ 的更高精度估计。)

Why does Richardson extrapolation improve the trapezoidal rule? / 为什么 Richardson 外推能改进梯形法?

"If the leading error in the trapezoidal rule is $O(h^2)$, then $I_h = I + ah^2$ and $I_{h/2} = I + ah^2/4$. Combining $4I_{h/2} - I_h$ cancels the leading error term, so $(4I_{h/2} - I_h)/3$ gives a higher-order

$O(h^4)$ estimate."

(如果 trapezoidal rule 的主导误差是 $O(h^2)$, 那么 $I_h = I + ah^2$, $I_{h/2} = I + ah^2/4$ 。组合 $4I_{h/2} - I_h$ 可以抵消主导误差项, 所以 $(4I_{h/2} - I_h)/3$ 给出更高阶的 $O(h^4)$ 估计。)

Why use non-uniform discretisation? / 为什么使用非均匀离散化?

"When a function changes much faster in some regions than others, more sub-intervals can be placed in rapidly varying regions and fewer in smoother regions. This achieves similar accuracy with fewer total function evaluations, and is called adaptive or non-uniform quadrature."

(当函数在某些区域变化远快于其他区域时, 可以在变化快的区域放更多子区间, 在变化慢的区域放更少子区间。这样能用更少的总函数计算次数达到类似精度, 称为 adaptive 或 non-uniform quadrature。)

Chapter 4: ODE Solvers 常微分方程数值解

识别标志

The exam gives $dy/dt = f(y,t)$, initial condition $y(0)$, asks for FE/BE/Heun/IT and convergence analysis. (题目给 ODE、初值、终点时间, 让你实现 Euler/Heun/Implicit 等方法并做收敛分析。)

真题原文

2020 Q2.2: " $dy/dt = y + t^3$, $y(0) = 1$... forward Euler and Heun's method up to $t=3$..."

2023 Q2: " $du/dt = u$, $u(0) = 1$... forward Euler... Heun... backward Euler... implicit trapezoidal..."

2024 Q2: "Explain why we need to discretise... solving ODEs numerically..."

2025: ODE not tested directly in 2025, but discretisation concept in Q2.1.

题目会给什么

- $f(y, t)$ (e.g. $dy/dt = y + t^3$, $du/dt = u$)
- $y(0)$ (e.g. $y(0) = 1$)
- End time (e.g. $t = 3$, $t = 5$)
- Exact solution (e.g. $y(t) = 7e^t - t^3 - 3t^2 - 6t - 6$)

General Linear ODE Rearrangement / 一般线性 ODE 显式整理

For **linear ODEs** (like $du/dt = u$), you can rearrange Backward Euler and IT algebraically so you do NOT need an iterative solver inside each time step.

(对于线性 ODE, Backward Euler 和 Implicit Trapezoidal 可以显式整理, 不需要每步内部迭代。)

If $dy/dt = a*y + g(t)$:

- BE:** $y_{n+1} = (y_n + dt*g(t_{n+1})) / (1 - a*dt)$
- IT:** $y_{n+1} = [y_n + (dt/2)*(a*y_n + g(t_n) + g(t_{n+1}))] / (1 - a*dt/2)$

Worked Example (2023 Q2) / 例题

Given: $du/dt = u$, $u(0) = 1$, exact $u(t) = \exp(t)$, solve to $t = 5$.

代码块

```
1 def solve_ode(method, f, y0, t0, tf, dt):
2     """
3     method: 'FE', 'BE', 'Heun', or 'IT'
4     f: f(y,t) [REPLACE: copy RHS from exam, e.g. lambda y,t: y + t**3]
5     y0: initial [REPLACE: copy from exam]
6     t0, tf: time range [REPLACE: copy from exam]
7     """
8     num_steps = int(round((tf - t0) / dt))
9     t = np.linspace(t0, tf, num_steps + 1)
10    y = np.zeros_like(t)
11    y[0] = y0
12
13    for n in range(len(t) - 1):
14        y_n = y[n]
15
16        if method == 'FE':
17            y[n+1] = y_n + dt * f(y_n, t[n])
18        elif method == 'BE':
19            # For linear du/dt = u: y_{n+1} = y_n / (1 - dt) / 线性方程
19            du/dt=u: 后向 Euler 显式解
20            y[n+1] = y_n / (1 - dt)
21        elif method == 'Heun':
22            y_tilde = y_n + dt * f(y_n, t[n])
23            y[n+1] = y_n + 0.5 * dt * (f(y_n, t[n]) + f(y_tilde, t[n+1]))
24        elif method == 'IT':
25            # For linear du/dt = u: y_{n+1} = y_n * (1 + dt/2) / (1 - dt/2)
25            / 线性方程 du/dt=u: 隐式梯形显式解
26            y[n+1] = y_n * (1 + dt/2) / (1 - dt/2)
27
28    return t, y
29
30 f = lambda y, t: y # [REPLACE: copy dy/dt = ... from exam / 替换: 从
30 考题复制 dy/dt]
31 y0 = 1.0 # [REPLACE: copy initial condition / 替换: 复制初
31 始条件]
32 t0, tf = 0.0, 5.0 # [REPLACE: copy time range / 替换: 复制时间范围]
33 exact_final = np.exp(5) # [REPLACE: exact solution at tf / 替换: tf 处的精
33 确解]
```

结果解读

--	--	--	--

Method	Ratio	Order	Best performance sentence / 表现最好时抄写
FE	≈ 2.0	1st $O(\Delta t)$	"Forward Euler systematically underestimates."
BE	≈ 2.0	1st $O(\Delta t)$	"Backward Euler systematically overestimates."
Heun	≈ 4.0	2nd $O(\Delta t^2)$	"Heun's method averages two slopes for better accuracy."
IT	≈ 4.0	2nd $O(\Delta t^2)$	"Implicit Trapezoidal performs best: smallest error, excellent stability."

文字题模板

Why is the Trapezoidal method closer to the exact solution? / 为什么 Trapezoidal method 更接近精确解?

"Forward Euler's trajectory falls below the exact solution, so it underestimates the result, while Backward Euler lies above it and overestimates the result. The Trapezoidal method averages these two slopes, producing a trajectory much closer to the exact solution."

(Forward Euler 的轨迹低于 exact solution, 因此会 underestimate; Backward Euler 的轨迹高于 exact solution, 因此会 overestimate。Trapezoidal method 对这两个斜率取平均, 所以得到的轨迹更接近 exact solution。)

Chapter 5: Root Finding 方程求根

识别标志

The exam gives $f(x)$ and asks for roots using bracketing, bisection, Newton, secant, or demonstrate failure modes.

(题目给 $f(x)$, 让你用 bracketing、bisection、Newton、secant 求根, 并演示失效。)

真题原文

2021 Q3: "root-finding problem applied to quadratic $f(x)=ax^2+bx+c...$ use Newton's method... explain and demonstrate how Newton's method can fail."

2022 Q4: " $f(x) = 1/((x-0.2)^2+0.1) - 1/((x-0.6)^2+0.05)$... bracketing... bisection, quasi-Newton, secant... Newton..."

2023 Q4: "cubic function $f(x)=ax^3+bx^2+cx+d$... bracketing... Newton... explain failure..."

2024 Q4: "linear $f_1(x)=ax+b$, quadratic $f_2(x)=ax^2+bx+c$, cubic $f_3(x)=ax^3+bx^2+cx+d$... 0/1/2/3 real roots..."

2025 Q4: " $f(x) = 5x \cdot \exp(-x^2) - \cos(x)$... bracketing... two methods... compare iterations... demonstrate Newton failure..."

题目会给什么

- $f(x)$ (e.g. cubic, Lorentzian, or $5x \cdot \exp(-x^2) - \cos(x)$)
- Sometimes derivative formula (e.g. 2025 Q4 gives $f'(x)$)
- Interval or initial guess

Worked Example (2025 Q4) / 例题

Given: $f(x) = 5x \cdot \exp(-x^2) - \cos(x)$ over $[-2, 2]$.

Part A: Bracketing / 变号区间

▼ 代码块

Python ▾ | ↵ 取消自动换行 | 复制

```
1 f = lambda x: 5*x*np.exp(-x**2) - np.cos(x) # [REPLACE: copy from exam / 替
    换: 从考题复制]
2
3 ▼ def find_all_brackets(f, a, b, dx):
4     x = np.arange(a, b + dx, dx)
5     brackets = []
6 ▼     for i in range(len(x)-1):
7 ▼         if f(x[i]) * f(x[i+1]) < 0:
8             brackets.append((x[i], x[i+1]))
9     return brackets
10
11 brackets = find_all_brackets(f, -2, 2, 0.1)
12 print("Brackets:", brackets)
```

Part B1: Bisection + Newton / 二分法 + 牛顿法

代码块

```
1 def bisection(f, a, b, tol=1e-6):
2     if f(a)*f(b) > 0:
3         raise ValueError("No sign change / 区间端点没有变号")
4     for i in range(100):
5         c = (a+b)/2
6         if f(c) == 0: # exact root found / 找到精确根
```

```

7         return c, i
8     elif f(a)*f(c) < 0: b = c
9     else: a = c
10    if b-a < tol: return c, i+1
11    return c, 100
12
13 def newton(f, df, x0, tol=1e-6):
14     x = x0
15     for i in range(50):
16         fx, dfx = f(x), df(x)
17         if abs(dfx) < 1e-14:
18             raise RuntimeError("Derivative too small / 导数太小, 方法失效")
19         x_new = x - fx/dfx
20         if abs(x_new-x) < tol: return x_new, i+1
21         x = x_new
22     raise RuntimeError("Did not converge / 未收敛")
23
24 # 2025 Q4 gives derivative: / 2025 Q4 给出导数:
25 df = lambda x: 5*np.exp(-x**2)*(1 - 2*x**2) + np.sin(x)
26
27 # Choose one root / 选一个根
28 root_b, it_b = bisection(f, -0.5, 0.5)    # [REPLACE: your bracket / 替换: 你
    的区间]
29 root_n, it_n = newton(f, df, -0.5)        # [REPLACE: your initial guess / 替
    换: 你的初始猜测]
30 print(f"Bisection: {root_b:.6f} in {it_b} iterations")
31 print(f"Newton:      {root_n:.6f} in {it_n} iterations")

```

Part B2: Secant Method (2022 Q4.2) / 割线法

Secant does not need the derivative; it approximates it from two previous points.

(割线法不需要导数, 用两个前点近似导数。)

代码块

```

1 def secant(f, x0, x1, tol=1e-6):
2     x = [x0, x1]    # [REPLACE: your initial guesses / 替换: 你的两个初始猜测]
3     for i in range(100):
4         dfdx = (f(x[-1]) - f(x[-2])) / (x[-1] - x[-2])
5         x.append(x[-1] - f(x[-1]) / dfdx)
6         if abs(x[-1] - x[-2]) < tol:
7             return x[-1], i+1
8     return x[-1], 100
9
10 root_s, it_s = secant(f, -0.5, 0.0)
11 print(f"Secant: {root_s:.6f} in {it_s} iterations")

```

Part B3: Quasi-Newton Method (2022 Q4.2) / 拟牛顿法

Quasi-Newton approximates the derivative with a finite difference step dx .

(拟牛顿法用有限差分步长 dx 近似导数。)

代码块

```
1 def quasi_newton(f, x0, dx=1e-7, tol=1e-6):
2     x = [x0] # [REPLACE: your initial guess / 替换: 你的初始猜测]
3     for i in range(100):
4         dfdx = (f(x[-1] + dx) - f(x[-1])) / dx
5         x.append(x[-1] - f(x[-1]) / dfdx)
6         if abs(x[-1] - x[-2]) < tol:
7             return x[-1], i+1
8     return x[-1], 100
9
10 root_qn, it_qn = quasi_newton(f, -0.5)
11 print(f"Quasi-Newton: {root_qn:.6f} in {it_qn} iterations")
```

Part C: Newton Failure Demonstration (2025 Q4.3) / 演示 Newton 失效

代码块

```
1 # Failure 1: poor initial guess converges to wrong root / 初值不好→非最近根
2 root_wrong, _ = newton(f, df, 2.0) # [REPLACE: far from desired root / 替
   换: 远离目标根的初值]
3 print(f"Starting from 2.0, converged to {root_wrong:.4f}")
4
5 # Failure 2: derivative near zero → overflow / 导数接近零→溢出
6 try:
7     x_bad = 1.0 # [REPLACE: point where df/dx ≈ 0 / 替换: 导数近似为零的点]
8     root_bad, _ = newton(f, df, x_bad)
9 except RuntimeError as e:
10     print("Failure:", e)
11
12 # Plot / 画图
13 x = np.linspace(-2, 2, 400)
14 plt.plot(x, f(x), 'b-', label='f(x)')
15 plt.axhline(0, color='k', lw=0.5)
16 plt.legend(); plt.xlabel('x'); plt.ylabel('f(x)')
17 plt.show()
```

更优算法: Picard Iteration / Picard 迭代

Picard iteration rewrites $f(x)=0$ as $x = g(x)$ and iterates $x_{n+1} = g(x_n)$.

(Picard 迭代将 $f(x)=0$ 改写为 $x=g(x)$, 然后迭代 $x_{n+1}=g(x_n)$ 。)

```

1 def picard(g, x0, tol=1e-6, max_iter=100):
2     """Picard iteration  $x_{n+1} = g(x_n)$  / Picard 迭代"""
3     x = x0
4     for i in range(max_iter):
5         x_new = g(x)
6         if abs(x_new - x) < tol:
7             return x_new, i+1
8         x = x_new
9     return x, max_iter
10
11 # Example: solve  $x = \cos(x)$  / 示例: 解  $x = \cos(x)$ 
12 g = lambda x: np.cos(x)
13 root, iters = picard(g, 0.5)
14 print(f"Picard root: {root:.6f} in {iters} iterations")

```

文字题模板

How can Newton's method fail? / Newton 方法如何失效?

"Newton's method can fail in two common ways. First, if $f'(x_n)$ is very close to zero, the correction term becomes very large, so the next iterate may jump far away and cause divergence or overflow. Second, if the initial guess is poor, the method may converge to a different root instead of the desired one. A practical remedy is to first bracket the root using bisection, and then switch to Newton's method for rapid refinement."

(Newton 方法常见有两种失效方式。第一，如果 $f'(x_n)$ 很接近 0，修正项会变得很大，导致下一步迭代跳得很远，从而引起 divergence 或 overflow。第二，如果 initial guess 不够好，方法可能收敛到另一个根，而不是目标根。一个常见的 fix 是先用 bisection bracket root，再切换到 Newton 方法进行快速 refinement。)

How many real roots can a cubic have? / 三次函数可以有几个实根?

"A cubic polynomial always has at least one real root and can have up to three real roots. By comparison, a linear function has 0 or 1 real root, and a quadratic has 0, 1, or 2 real roots. For a cubic, the exact number of real roots depends on the coefficients, or equivalently on the discriminant."

(三次多项式一定至少有 1 个 real root，最多可以有 3 个 real roots。作为对比，linear function 有 0 或 1 个 real root，quadratic function 有 0、1 或 2 个 real roots。对于 cubic，实根的具体个数取决于系数，也可以说取决于 discriminant。)

What is Picard iteration? / 什么是 Picard iteration?

"Picard iteration rewrites $f(x)=0$ into the fixed-point form $x=g(x)$, then iterates using $x_{n+1}=g(x_n)$. It converges if $|g'(x)|<1$ near the root. The method is simple, but it may diverge if this slope condition is not satisfied."

(Picard iteration 是把 $f(x)=0$ 改写成 fixed-point form: $x=g(x)$ ，然后用 $x_{n+1}=g(x_n)$ 迭代。若在根附近 $|g'(x)|<1$ ，则方法收敛。这个方法简单，但如果不满足 slope condition，可能会 diverge。)

Chapter 6: Linear Systems — Hand Calculation 线性系统手算

识别标志

The exam gives a 3×3 system, asks for matrix form, augmented matrix, row operations by hand, back substitution, and LU decomposition.

(题目给 3×3 方程组，让你写成矩阵形式、构造增广矩阵、手算行变换、回代、LU 分解。)

真题原文

2022 Q5: "Form $[A|b]$... transform to upper-triangular... back substitution (do this all with pen and paper)..."

2023 Q5: "Form $[A|b]$... transform to upper-triangular... back substitution (do this 'by hand')..."

2024 Q5: "Form $[A|b]$... upper-triangular... back substitution (by hand)... also transform to lower-triangular and use forward substitution..."

2025 Q5: "Write in matrix form $Ax=b$... form $[A|b]$... row operations to upper-triangular... back substitution... LU decomposition... verify $LU=A$..."

Worked Example (2025 Q5)

Given:

代码块

$$\begin{array}{lcl} 1 & 2x + y - z & = 8 \\ 2 & -3x - y + 2z & = -11 \\ 3 & -2x + y + 2z & = -3 \end{array}$$

Step 1: Matrix form / 矩阵形式

代码块

$$\begin{array}{lcl} 1 & A = [2 & 1 & -1] & b = [8] \\ 2 & & [-3 & -1 & 2] & [-11] \\ 3 & & [-2 & 1 & 2] & [-3] \end{array}$$

Step 2: Augmented matrix / 增广矩阵

代码块

$$\begin{array}{lcl} 1 & [2 & 1 & -1 & | & 8] \\ 2 & [-3 & -1 & 2 & | & -11] \end{array}$$

```
3  [-2  1  2 | -3 ]
```

Step 3: Forward elimination (upper-triangular) / 前向消元

Column 1 (pivot = 2 / 主元=2):

- $R2 \leftarrow R2 - (-3/2) \times R1 = R2 + 1.5 \times R1$
 - $[-3+3, -1+1.5, 2-1.5 \mid -11+12] = [0, 0.5, 0.5 \mid 1]$
- $R3 \leftarrow R3 - (-2/2) \times R1 = R3 + 1 \times R1$
 - $[-2+2, 1+1, 2-1 \mid -3+8] = [0, 2, 1 \mid 5]$

Intermediate:

代码块

```
1  [ 2  1 -1 | 8 ]
2  [ 0  0.5 0.5 | 1 ]
3  [ 0  2  1 | 5 ]
```

Column 2 (pivot = 0.5 / 主元=0.5):

- $R3 \leftarrow R3 - (2/0.5) \times R2 = R3 - 4 \times R2$
 - $[0, 2-2, 1-2 \mid 5-4] = [0, 0, -1 \mid 1]$

Upper-triangular:

代码块

```
1  [ 2  1 -1 | 8 ]
2  [ 0  0.5 0.5 | 1 ]
3  [ 0  0 -1 | 1 ]
```

Step 4: Back substitution / 回代

- R3: $-z = 1 \rightarrow \mathbf{z = -1}$
- R2: $0.5y + 0.5(-1) = 1 \rightarrow 0.5y = 1.5 \rightarrow \mathbf{y = 3}$
- R1: $2x + 3 - (-1) = 8 \rightarrow 2x = 4 \rightarrow \mathbf{x = 2}$

Step 5: Verify / 验证

代码块

```
1  A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
2  b = np.array([8, -11, -3], dtype=float)
3  print(np.linalg.solve(A, b))    # [2.  3. -1.]
```

Worked Example / Verify by LU = A

From the elimination above:

- $m_{21} = -3/2 = -1.5 \rightarrow \mathbf{L[2,1] = -1.5}$
- $m_{31} = -2/2 = -1 \rightarrow \mathbf{L[3,1] = -1}$
- $m_{32} = 2/0.5 = 4 \rightarrow \mathbf{L[3,2] = 4}$

代码块

```
1  L = [ 1   0   0 ]      U = [ 2   1  -1 ]
2      [-1.5 1   0 ]      [ 0 0.5 0.5 ]
3      [-1   4   1 ]      [ 0   0  -1 ]
```

Compute each entry of LU: / 逐元素计算 LU:

- (1,1): $1 \times 2 + 0 \times 0 + 0 \times 0 = \mathbf{2}$
- (1,2): $1 \times 3 + 0 \times (-1) + 0 \times 0 = \mathbf{3}$
- (1,3): $1 \times (-4) + 0 \times 14 + 0 \times 30 = \mathbf{-4}$
- (2,1): $3 \times 2 + 1 \times 0 + 0 \times 0 = \mathbf{6}$
- (2,2): $3 \times 3 + 1 \times (-1) + 0 \times 0 = 9 - 1 = \mathbf{8}$
- (2,3): $3 \times (-4) + 1 \times 14 + 0 \times 30 = -12 + 14 = \mathbf{2}$
- (3,1): $2 \times 2 + (-2) \times 0 + 1 \times 0 = \mathbf{4}$
- (3,2): $2 \times 3 + (-2) \times (-1) + 1 \times 0 = 6 + 2 = \mathbf{8}$
- (3,3): $2 \times (-4) + (-2) \times 14 + 1 \times 30 = -8 - 28 + 30 = \mathbf{-6}$

Result matches original A. / 结果与原矩阵 A 完全一致。

Matrix Inversion via [A | I] / 增广矩阵求逆法

If the exam asks you to **compute A^{-1} using row operations**, form the augmented matrix $[A | I]$ and perform Gaussian elimination to transform the left side into the identity matrix. The right side then becomes A^{-1} .

(如果题目要求用手算行变换求逆矩阵，构造增广矩阵 $[A|I]$ ，对左边做高斯消元化为单位矩阵，右边即为 A^{-1} 。)

Worked Example / 手算例题 (2022 Q5.3):

Given / 题目给出:

代码块

```
1  A = [ 1   2  -2 ]
2      [-1   0   8 ]
```

Step 1: Form $[A | I]$ / 构造增广矩阵

代码块

```
1  [ 1  2 -2 | 1  0  0 ]
2  [-1  0  8 | 0  1  0 ]
3  [ 2  6  3 | 0  0  1 ]
```

Step 2: Forward elimination (left side $\rightarrow$ upper triangular) / 前向消元

Eliminate column 1:

- $R2 \leftarrow R2 + R1$ (multiplier $m_{21} = -1/1 = -1$, so $R2 \leftarrow R2 - (-1)R1$)
 - $[-1+1, 0+2, 8+(-2) | 0+1, 1+0, 0+0] = [0, 2, 6 | 1, 1, 0]$
- $R3 \leftarrow R3 - 2 \times R1$ (multiplier $m_{31} = 2/1 = 2$)
 - $[2-2, 6-4, 3-(-4) | 0-2, 0-0, 1-0] = [0, 2, 7 | -2, 0, 1]$

Result after column 1: / 第1列消元后:

代码块

```
1  [ 1  2 -2 | 1  0  0 ]
2  [ 0  2  6 | 1  1  0 ]
3  [ 0  2  7 | 0  0  1 ]
```

Eliminate column 2:

- $R3 \leftarrow R3 - R2$ (multiplier $m_{32} = 2/2 = 1$)
 - $[0, 2-2, 7-6 | -2-1, 0-1, 1-0] = [0, 0, 1 | -3, -1, 1]$

Upper-triangular form: / 上三角形式:

代码块

```
1  [ 1  2 -2 | 1  0  0 ]
2  [ 0  2  6 | 1  1  0 ]
3  [ 0  0  1 | -1 -1  1 ]
```

Step 3: Back elimination (left side $\rightarrow$ diagonal) / 后向消元

Eliminate above diagonal in column 3:

- $R2 \leftarrow R2 - 6 \times R3$
 - $[0, 2, 6-6 | 1-6 \times (-3), 1-6 \times (-1), 0-6 \times 1] = [0, 2, 0 | 19, 7, -6]$

- $R1 \leftarrow R1 + 2 \times R3$
 - $[1, 2, -2+2 \mid 1+2 \times (-3), 0+2 \times (-1), 0+2 \times 1] = [1, 2, 0 \mid -5, -2, 2]$

Result: / 结果:

代码块

```
1  [ 1  2  0 | -1  -2  2 ]
2  [ 0  2  0 |  7   7 -6 ]
3  [ 0  0  1 | -1  -1  1 ]
```

Eliminate above diagonal in column 2:

- $R1 \leftarrow R1 - R2$
 - $[1, 2-2, 0 \mid -5-19, -2-7, 2-(-6)] = [1, 0, 0 \mid -24, -9, 8]$

Diagonal form: / 对角形式:

代码块

```
1  [ 1  0  0 | -8  -9  8 ]
2  [ 0  2  0 |  7   7 -6 ]
3  [ 0  0  1 | -1  -1  1 ]
```

Step 4: Scale rows to make left side = I / 归一化

- $R1$: divide by 1 $\rightarrow [1, 0, 0 \mid -8, -9, 8]$
- $R2$: divide by 2 $\rightarrow [0, 1, 0 \mid 3.5, 3.5, -3]$
- $R3$: divide by 1 $\rightarrow [0, 0, 1 \mid -1, -1, 1]$

Final $[I \mid A^{-1}]$: / 最终结果:

代码块

```
1  [ 1  0  0 | -8  -9  8 ]
2  [ 0  1  0 | 3.5  3.5 -3 ]
3  [ 0  0  1 | -1  -1  1 ]
```

Therefore / 因此:

代码块

```
1  A-1 = [ -24  -9   8 ]
2          [  9.5  3.5 -3 ]
3          [  -3   -1   1 ]
```

Step 5: Verify with Python / 用 Python 验证

代码块

```
1 A = np.array([[1., 2., -2.],
2               [-1., 0., 8.],
3               [2., 6., 3.]])
4 print("Inverse:", np.linalg.inv(A))
5 # Output matches hand calculation above. / 输出与手算结果一致。
```

Worked Example / Forward Substitution (2024 Q5.1, 2025 Q5.2)

If you form **lower-triangular** L (by eliminating from bottom up):

代码块

```
1 [ 2  1 -1 | 8 ]
2 [-3 -1  2 |-11]
3 [-2  1  2 |-3 ]
```

Step 1: Eliminate column 3 (pivot = R3) / 第3列消元

- $R2 \leftarrow R2 - (2/2)R3 = R2 - R3$
 - $[-3-(-2), -1-1, 2-2 \mid -11-(-3)] = [-1, -2, 0 \mid -8]$
- $R1 \leftarrow R1 - (-1/2)R3 = R1 + 0.5R3$
 - $[2+0.5(-2), 1+0.5(1), -1+0.5(2) \mid 8+0.5(-3)] = [1, 1.5, 0 \mid 6.5]$

Intermediate / 中间矩阵

代码块

```
1 [ 1  1.5  0 | 6.5 ]
2 [-1 -2   0 | -8 ]
3 [-2  1   2 | -3 ]
```

Step 2: Eliminate column 2 (pivot = R2) / 第2列消元

- $R1 \leftarrow R1 - (1.5/(-2))R2 = R1 + 0.75R2$
 - $[1+0.75(-1), 1.5+0.75(-2), 0 \mid 6.5+0.75(-8)] = [0.25, 0, 0 \mid 0.5]$

Lower-triangular form / 下三角形形式

代码块

```
1 [ 0.25  0  0 | 0.5 ]
2 [-1    -2  0 | -8 ]
```

```
3  [-2    1    2 | -3  ]
```

Step 3: Normalise diagonal to 1 / 对角线归一为1

- $R1 \leftarrow R1 / 0.25 = [1, 0, 0 | 2]$
- $R2 \leftarrow R2 / (-2) = [0.5, 1, 0 | 4]$
- $R3 \leftarrow R3 / 2 = [-1, 0.5, 1 | -1.5]$

Final $[L | c]$ / 最终增广矩阵

代码块

```
1  [ 1    0    0 |  2  ]
2  [ 0.5  1    0 |  4  ]
3  [-1    0.5  1 | -1.5 ]
```

Step 4: Forward substitution (solve $Lc = b$) / 前代求解

- $c_1 = 2 / 1 = 2$
- $c_2 = (4 - 0.5 \times 2) / 1 = 3$
- $c_3 = (-1.5 - (-1) \times 2 - 0.5 \times 3) / 1 = (-1.5 + 2 - 1.5) = -1$

Solution / 解

代码块

```
1  x = [ 2,  3, -1 ]
```

Step 5: Verify with Python / 用 Python 验证

代码块

```
1  A = np.array([[2, 1, -1], [-3, -1, 2], [-2, 1, 2]], dtype=float)
2  b = np.array([8, -11, -3], dtype=float)
3  print("Solution:", np.linalg.solve(A, b))
4  # Output: [ 2.  3. -1.] / 输出与手算结果一致。
```

What is the key principle for finding an inverse matrix? / 求逆矩阵的核心原理是什么?

"Every elementary row operation performed on A must be performed simultaneously on I. When A is transformed into I, the original identity matrix I has been transformed into A^{-1} ."

(对 A 做的每一个 elementary row operation, 都必须同步施加在 I 上。当 A 被变成 I 时, 原来的 identity matrix I 就被变成了 A^{-1} 。)

Why is LU decomposition useful? / 为什么 LU 分解有用?

"After decomposing A into L and U , solving $Ax = b$ becomes much faster because we only need two triangular substitution steps: first solve $Lc = b$ by forward substitution, then solve $Ux = c$ by back substitution. This is useful when the same coefficient matrix A is used with several different right-hand-side vectors, because the LU decomposition only needs to be computed once."

(将 A 分解为 L 和 U 之后, 求解 $Ax = b$ 会更快, 因为只需要两步三角替代: 先用 forward substitution 解 $Lc = b$ 得到 c , 再用 back substitution 解 $Ux = c$ 得到 x 。当同一个系数矩阵 A 要对应多个不同的 right-hand-side vectors 时, 这很有用, 因为 LU decomposition 只需要计算一次。)

What is partial pivoting? / 什么是 partial pivoting?

"Partial pivoting swaps rows during Gaussian elimination to move the largest available element in the current column into the pivot position. This avoids division by very small pivot values, which would otherwise amplify round-off errors and lead to an unreliable solution."

(Partial pivoting 是在 Gaussian elimination 中交换行, 把当前列中最大的可用元素移动到 pivot position。这样可以避免除以非常小的 pivot value, 否则会放大 round-off errors, 导致解不可靠。)

Chapter 7: Iterative Methods & Condition Number 迭代法与条件数

识别标志

The exam asks about Jacobi vs Gauss-Seidel, condition number, or SOR/omega effects.

(题目让你比较 Jacobi 与 Gauss-Seidel、解释条件数、讨论 SOR/omega。)

真题原文

2021 Q4.3: "Explain differences between direct and iterative... Solve using Gauss-Seidel..."

Jacobi vs Gauss-Seidel / Jacobi 与 Gauss-Seidel 对比

代码块

```
1 def jacobi(A, b, x0=None, tol=1e-6, maxit=500):
2     n = len(b)
3     x = x0.copy() if x0 is not None else np.zeros(n)
4     for it in range(maxit):
5         x_new = np.zeros_like(x)
6         for i in range(n):
7             sigma = sum(A[i,j] * x[j] for j in range(n) if j != i)
8             x_new[i] = (b[i] - sigma) / A[i,i]
9         if np.linalg.norm(x_new - x, ord=np.inf) < tol:
10             return x_new, it+1
11     x = x_new
12     return x, maxit
```

```

13
14 def gauss_seidel(A, b, x0=None, tol=1e-6, maxit=500):
15     n = len(b)
16     x = x0.copy() if x0 is not None else np.zeros(n)
17     for it in range(maxit):
18         x_old = x.copy()
19         for i in range(n):
20             sigma = sum(A[i,j] * x[j] for j in range(n) if j != i)
21             x[i] = (b[i] - sigma) / A[i,i]
22         if np.linalg.norm(x - x_old, ord=np.inf) < tol:
23             return x, it+1
24     return x, maxit

```

Condition Number / 条件数

代码块

```

1 A = np.array([[2, 1], [2, 1.001]], dtype=float)
2 print(f"cond(A) = {np.linalg.cond(A):.2f}")

```

cond(A)	Sentence to copy / 抄写	中文
$\approx 1-10$	"Well-conditioned. Small perturbations produce small changes."	良态
$\gg 1$ (e.g. >1000)	"Ill-conditioned. Tiny changes cause huge changes in x."	病态
∞	"Singular. No inverse, no unique solution."	奇异

SOR, Weighted Jacobi and Convergence Conditions / SOR、加权 Jacobi 与收敛条件

SOR = Successive Over-Relaxation

ω value / 取值	Effect / 效果	中文说明
$\omega = 0$	No update. The iteration stays at the initial guess.	不更新，保持初值，系统未求解
$\omega = 1$	Standard Jacobi or Gauss-Seidel iteration.	标准 Jacobi/GS

$\omega > 1$	Over-relaxation. Often accelerates convergence.	超松弛，通常加速收敛
$0 < \omega < 1$	Under-relaxation. Usually slows convergence.	欠松弛，通常减慢收敛

更优算法：Frobenius Norm / Frobenius 范数

The Frobenius norm is the matrix analogue of the Euclidean vector norm: $\|A\|_F = \sqrt{(\sum |a_{ij}|^2)}$.
(Frobenius 范数是矩阵的欧几里得范数：所有元素平方和再开根号。)

代码块

```
1 def frobenius_norm(A):
2     """Frobenius norm by hand / 手算 Frobenius 范数"""
3     return np.sqrt(np.sum(A**2))
4
5 # Or use numpy / 或用 numpy
6 print(np.linalg.norm(A, 'fro'))
```

更优算法：Diagonal Dominance Test / 对角占优测试

Strict diagonal dominance guarantees convergence of Jacobi and Gauss-Seidel.
(严格对角占优保证 Jacobi 和 Gauss-Seidel 收敛。)

代码块

```
1 def is_diag_dom(A):
2     """Check strict diagonal dominance / 检查严格对角占优"""
3     n = len(A)
4     for i in range(n):
5         off_diag = np.sum(np.abs(A[i])) - abs(A[i,i])
6         if off_diag >= abs(A[i,i]):
7             return False
8     return True
9
10 # [REPLACE: copy matrix from exam / 替换: 从考题复制矩阵]
11 print("Diagonally dominant:", is_diag_dom(A))
```

更优算法：Hilbert & Vandermonde Matrices / Hilbert 与 Vandermonde 矩阵

Classic examples of ill-conditioned matrices. The Hilbert matrix has entries $H[i, j] = 1/(i + j + 1)$.
The Vandermonde matrix has entries $V[i, j] = x_i^j$.

(经典病态矩阵例子。Hilbert 矩阵元素 $H[i, j] = 1/(i + j + 1)$ ，Vandermonde 矩阵元素 $V[i, j] = x_i^j$ 。)

```

1 def hilbert(n):
2     """Generate n×n Hilbert matrix / 生成 Hilbert 矩阵"""
3     A = np.zeros((n, n))
4     for i in range(n):
5         for j in range(n):
6             A[i,j] = 1.0 / (i + j + 1)
7     return A
8
9 def vandermonde(alpha_vec):
10    """Generate Vandermonde matrix / 生成 Vandermonde 矩阵"""
11    n = len(alpha_vec)
12    V = np.zeros((n, n))
13    for i in range(n):
14        for j in range(n):
15            V[i,j] = alpha_vec[i]**j
16    return V
17
18 # Demonstrate ill-conditioning / 演示病态
19 for n in range(2, 15):
20     H = hilbert(n)
21     print(f"n={n:2d}: cond(H) = {np.linalg.cond(H):.2e}")

```

文字题模板

Direct vs Iterative Methods / 直接法 vs 迭代法

"Direct methods produce the exact solution up to round-off error in a finite number of operations. Iterative methods generate a sequence of approximations that converge toward the true solution, with final accuracy controlled by a residual tolerance. Gauss-Seidel is generally preferred over Jacobi because it uses updated values immediately."

(Direct methods 在有限步运算内得到精确解，误差主要来自 round-off error。Iterative methods 生成一系列逐步接近真实解的近似值，最终精度由 residual tolerance 控制。Gauss-Seidel 通常优于 Jacobi，因为它会立即使用更新后的值。)

Why does Gauss-Seidel converge faster than Jacobi? / 为什么 Gauss-Seidel 比 Jacobi 收敛更快?

"Gauss-Seidel usually converges faster than Jacobi because it immediately uses the most recently updated component values within the same iteration, instead of waiting until the next iteration."

(Gauss-Seidel 通常比 Jacobi 收敛更快，因为它会在同一轮迭代中立即使用最新更新的分量值，而不是等到下一轮迭代才使用。)

What is the condition number? / 什么是条件数?

"The condition number is defined as $\kappa(A) = \|A\| \|A^{-1}\|$. If $\kappa \approx 1$, the matrix is well-conditioned. If $\kappa \gg 1$, the matrix is ill-conditioned, meaning tiny round-off errors can cause

very large changes in the solution x . Common examples include the Hilbert matrix and Vandermonde matrix."

(Condition number 定义为 $\kappa(A) = \|A\| \|A^{-1}\|$ 。如果 $\kappa \approx 1$ ，矩阵是 well-conditioned；如果 $\kappa \gg 1$ ，矩阵是 ill-conditioned，说明很小的 round-off errors 也可能导致解 x 发生很大变化。常见例子包括 Hilbert matrix 和 Vandermonde matrix。)

What is the diagonal dominance test? / 什么是对角占优测试?

"A matrix is strictly diagonally dominant if the absolute value of each diagonal entry is greater than the sum of the absolute values of the other entries in the same row. If a matrix is strictly diagonally dominant, both Jacobi and Gauss-Seidel methods are guaranteed to converge."

(如果每一行中，对角线元素的绝对值都大于该行其他元素绝对值之和，则矩阵是 strictly diagonally dominant。如果矩阵严格对角占优，那么 Jacobi 和 Gauss-Seidel methods 都保证收敛。)

附录：真题题型速查

Year	Q1	Q2	Q3	Q4	Q5
2020	Interp + Fitting	Diff + ODE	Integration (Composite)	Linear Algebra (LU)	—
2021	Diff + ODE + Taylor	Integration (Simpson proof)	Interp + Root Finding	Linear Algebra (LU + GS)	—
2022	Interp Theory	Diff (4th order)	Integration Convergence	Root Finding	Linear Algebra (Elim + Inverse)
2023	Interp + Fitting Clouds	ODE (IT)	Integration (Weddle)	Root Finding (Cubic)	Linear Algebra (Elim + Algorithm)
2024	Interp (f vs g, N samples)	Diff (discretise + Taylor)	Integration (non-uniform)	Root Finding (1/2/3 roots)	Linear Algebra (Elim + Forward Sub)
2025	Interp (noisy data, MSE)	Diff (Taylor derive + round- off)	Integration (x^2+1)	Root Finding ($5x \cdot e^{-x^2} - \cos(x)$)	Linear Algebra (Elim + LU)